



UNIVERSIDADE DE BRASÍLIA

Faculdade do Gama · Engenharia de Software

RELATÓRIO TÉCNICO · DIAGNÓSTICO DE PROJETO

MeasureSoftGram: estado do ecossistema

Avaliação técnica dos sete repositórios do projeto —
arquitetura, integridade do modelo de qualidade, segurança,
cobertura de testes e integração contínua.

DATA

25 de agosto de 2026

ESCOPO

7 repositórios

CICLO

2026.1

ORGANIZAÇÃO

fga-eps-mds

3 VERSÕES DO CORE EM CIRCULAÇÃO	5 ACHADOS DE SEGURANÇA	3 REPOSITÓRIOS CONGELADOS	92% COBERTURA DO CORE
0 ERROS DE LINT EM PYTHON			

SUMÁRIO

01	Panorama do ecossistema	
02	Ritmo de desenvolvimento	
03	Integridade do modelo de qualidade	
04	Saúde das suítes de teste	
05	Segurança da aplicação	
06	Integração e entrega contínuas	
07	Dívida técnica acumulada	
08	Pontos fortes	
09	Plano de ação priorizado	
10	Método e limitações	

01 **Panorama do ecossistema**

O MeasureSoftGram não é um programa único: é um ecossistema de módulos independentes distribuídos em repositórios separados. Hoje ele existe em duas versões paralelas que não coincidem.

Foram analisados **sete repositórios** — Action, CLI, Core, Docs, Front, Parser e Service — todos com árvore de trabalho limpa e apontando para os repositórios **centrais** da organização, sem prefixo de semestre.

A documentação oficial do projeto, escrita em julho de 2026, descreve algo diferente: lista **nove repositórios ativos**, todos com prefixo `2026.1-`, e inclui dois módulos que não existem no conjunto central — **AI** (servidor MCP) e **Plugin** (extensão para VS Code). A página de arquitetura já os trata como parte integrante do sistema.

ACHADO ESTRUTURAL

A documentação descreve um ecossistema de nove módulos hospedado nos repositórios de semestre, enquanto o trabalho de consolidação de 2026.1 migra para repositórios centrais. Os dois modelos coexistem sem que nenhum seja a fonte da verdade.

MÓDULOS DECLARADOS NA DOCUMENTAÇÃO DE ARQUITETURA			
MÓDULO	RESPONSABILIDADE	STACK	PRESENTE
Core	Núcleo de cálculo do modelo de qualidade	Python	SIM
Parser	Normaliza dados de entrada para o Core	Python	SIM
Service	Backend REST, orquestra cálculo e persiste	Django	SIM
Front	Interface web e dashboards	Next.js	SIM
CLI	Ferramenta de linha de comando <code>msggram</code>	Python	SIM
Action	Integração com pipelines de CI	TypeScript	SIM
AI	Servidor MCP para assistentes de IA	Python	AUSENTE
Plugin	Extensão para editor VS Code	TypeScript	AUSENTE

02 Ritmo de desenvolvimento

Três repositórios estão vivos, três estão congelados e um é novo. Há um **hiato de dezessete meses** — de setembro de 2024 a março de 2026 — comum a todos. O desenvolvimento retomou no ciclo 2026.1, mas alcançou apenas quatro módulos.

ATIVIDADE POR REPOSITÓRIO			
REPOSITÓRIO	ÚLTIMO COMMIT	COMMITTS TOTAIS	SITUAÇÃO
Parser	24/08/2026	186	ATIVO 28 commits em 2026
Service	24/08/2026	858	ATIVO modernização pesada
CLI	24/08/2026	849	SINCRONIZADO 1 commit real
Docs	11/07/2026	21	NOVO criado em jun/2026
Core	20/03/2026	533	CONGELADO desde set/2024
Front	20/03/2026	1227	CONGELADO desde set/2024
Action	20/09/2024	338	CONGELADO há ~2 anos

Perda de histórico nos merges

Os commits intitulados `Sincronização de repositório - 2026.1` — pull requests #60 no Service e #54 no CLI — são *squashes* de grande porte. Um único commit traz 2.065 linhas alteradas em 45 arquivos no CLI, e centenas de arquivos no Service.

Todo o trabalho de um semestre — autoria individual, motivação de cada mudança, possibilidade de bissecção — colapsa em um commit só. O padrão já havia ocorrido antes: quatro repositórios receberam merges equivalentes no mesmo dia, 20 de março de 2026.

03 Integridade do modelo de qualidade

Este é o achado mais grave do diagnóstico. O motor de cálculo publicado não corresponde ao código versionado, e os dois modos de uso do sistema produzem notas diferentes para o mesmo software.

O Core publicado diverge do Core versionado

O arquivo `pyproject.toml` do repositório Core declara a versão 1.5.3, e o código local corresponde a essa versão. No PyPI, porém, existe `msgram-core 1.5.4`, publicado a partir de código que não está neste repositório.

A diferença entre as duas versões foi verificada baixando ambos os pacotes e comparando-os. Não são mudanças cosméticas — são correções semânticas do modelo de qualidade:

DIFF · MSGRAM-CORE 1.5.3 → 1.5.4

```
# aggregated_normalized_measures.py - medida ci_feedback_time
-   gain_interpretation=1,
+   gain_interpretation=-1,      ← sentido da métrica invertido
-   max_threshold: float = 0.33,
+   max_threshold: float = 0.66,  ← limiar dobrado
+   if int(total_builds) == 0:
+       return NO_BUILDS_NEUTRAL_SCORE ← score neutro (issue #24)

# resources/constants.py
-   "metrics": ["endpoint_calls", "cpu_usage"]
+   "metrics": ["endpoint_calls", "memory_usage"]

# util/run_time_data_operations.py - bug de matriz não-quadrada
-   return abs(sum / (size * size))
+   return abs(sum / (size_1 * size_2))
```

CONSEQUÊNCIA

A nota de qualidade calculada hoje pelo ecossistema **não é reproduzível** a partir do código versionado no repositório central do Core. O modelo de qualidade, que é o produto do projeto, deixou de ser auditável.

Três versões do Core em circulação simultânea

SERVICE · PRODUÇÃO

1.4.9

Fluxo hospedado:
dashboards e histórico de
releases

CLI · INSTALAÇÃO LIMPA

1.5.4

Fluxo offline: cálculo local
via terminal

REPOSITÓRIO CORE

1.5.3

A suposta fonte da verdade
do modelo

A documentação de arquitetura afirma que os dois modos de uso “compartilham o mesmo motor de cálculo”. Na prática, o Service está duas versões *minor* atrás do CLI, e nenhum dos dois roda o código que está versionado.

O CLI já quebrou por causa disso

Restrição de versão frouxa quebra o CLI em instalação limpa

CRÍTICO

O CLI declara `msgram-core~=1.5.3`, restrição que resolve para 1.5.4. Com essa versão, **três testes unitários falham**: `test_calculate_github`, `test_calculate_csv_output` e `test_calculate_json_output` — retornando `0.9677...` onde se espera `0.0322...`, exatamente o efeito da inversão de `gain_interpretation`.

Fixando `msgram-core==1.5.3`, os **127 testes unitários passam sem falhas**. Qualquer instalação limpa do CLI hoje, incluindo a de CI, está vermelha. Além disso, as expectativas dos testes codificam a semântica antiga — e incorreta — de `ci_feedback_time`.

MeasureSoftGram-CLI/pyproject.toml · verificado por execução

04 Saúde das suítes de teste

RESULTADO DAS SUÍTES EXECUTADAS			
REPOSITÓRIO	RESULTADO	COBERTURA	OBSERVAÇÃO
Core	PASSA	92%	Requer <code>PYTHONPATH=src</code>
Parser	2 FALHAS	90%	Regressão de 24/08/2026
CLI	127 TESTES	89%	Somente com Core 1.5.3
Service	NÃO EXECUTADO	—	274 testes definidos; exige Postgres
Front	NÃO EXECUTADO	—	72 arquivos de teste
Action	NÃO EXECUTADO	—	7 arquivos de teste

A análise estática de estilo foi executada nos quatro repositórios Python: **flake8 retorna zero ocorrências em todos**. O código Python do projeto está limpo em termos de convenção.

O Parser está quebrado no branch de desenvolvimento

Renomeação incompleta derruba dois testes no `develop`

CRÍTICO

O commit `f9bc01c` — “test: remove prefixo de semestre nos mocks de repositório”, de 24 de agosto de 2026 — renomeou `2023-1-MeasureSoftGram-DOC` para `MeasureSoftGram-DOC` no arquivo de teste, mas **não atualizou a fixture correspondente**. As linhas 16 e 34 de `tests/mockfiles/expected_return_values.py` ainda contêm o nome antigo.

A alteração foi integrada ao `develop` pelo pull request #24 no mesmo dia. A correção exige trocar duas *strings*. O fato de ter passado indica que o **resultado da integração contínua não está sendo verificado antes do merge** — o que é particularmente notável num projeto cujo produto é medir qualidade de software.

`MeasureSoftGram-Parser/tests/mockfiles/expected_return_values.py:16,34`

05 Segurança da aplicação

Cinco achados no Service, um deles com exposição direta de dados pessoais em endpoint público.

Endpoints públicos por omissão, incluindo listagem de usuários

CRÍTICO

A configuração `REST_FRAMEWORK` define `DEFAULT_AUTHENTICATION_CLASSES`, mas não define `DEFAULT_PERMISSION_CLASSES`. O comportamento padrão do Django REST Framework nesse caso é `AllowAny`: todo viewset que não declara permissão explícita fica aberto.

O caso mais sério é `UserListViewSet`, roteado em `GET /users/` sem autenticação, que serializa `id`, `username`, `first_name`, `last_name` e `email` de todos os usuários cadastrados — enumeração completa de dados pessoais.

Estão igualmente abertos os viewsets de `goals`, `release_configuration`, `entity_trees`, `math_model`, `measures`, `subcharacteristics` e `ReleaseListAllModelViewSet`. A correção é de uma linha: declarar `IsAuthenticated` como padrão e marcar explicitamente com `AllowAny` os endpoints de *badge* que devem permanecer públicos.

`src/config/settings/base.py:205 · src/accounts/views.py:95 · src/accounts/urls.py:51`

Django 4.0.6 — sem suporte desde abril de 2023

CRÍTICO

A versão está fixada no `pyproject.toml` do Service. São três anos sem correções de segurança, rodando em produção sobre Python 3.12. A migração mínima recomendada é para a série 4.2 LTS.

`MeasureSoftGram-Service/pyproject.toml`

Token de acesso do GitHub armazenado em texto plano

ATENÇÃO

O modelo de usuário guarda `github_access_token` como `CharField(max_length=255)`. É um token OAuth de terceiros, sem cifragem, na tabela de usuários.

`src/accounts/models.py`

Produção sem TLS, com endurecimento desativado por padrão

ATENÇÃO

O módulo `production.py` é bem escrito e documenta honestamente suas escolhas, mas entrega tudo desligado: `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE` e `CSRF_COOKIE_SECURE` em `False`, e `SECURE_HSTS_SECONDS` em zero. A justificativa registrada é que o deploy fica atrás de proxy HTTP sem terminação TLS.

Combinado com `TokenAuthentication`, isso significa que **tokens de autenticação trafegam em claro**. As chaves de configuração já estão prontas e parametrizadas — basta haver TLS na borda para ativá-las.

`src/config/settings/production.py`

Configuração apontando para infraestrutura desativada

ATENÇÃO

`CSRF_TRUSTED_ORIGINS` tem como padrão um domínio Railway do ciclo 2023-2, e `CORS_ALLOWED_ORIGINS` aponta para um domínio Vercel de 2024-1. Além disso, `debug_toolbar` figura em `INSTALLED_APPS` e `MIDDLEWARE` do módulo base — não apenas no de desenvolvimento — e consta entre as dependências de produção.

`src/config/settings/base.py:34,111`

06 Integração e entrega contínuas

A infraestrutura de CI/CD do projeto opera em duas velocidades muito distintas.

Service — o padrão de referência

A modernização executada em 2026.1 é trabalho de qualidade profissional: gerenciamento de dependências com `uv` e `lockfile`, execução com `--frozen` e `--locked`, **actions fixadas por SHA**, Postgres 18 nos testes, verificação de `makemigrations --check`, Docker multi-estágio, deploy em *runner* auto-hospedado na máquina do LAPPIS acionado por `workflow_run`, com *healthcheck* e teste de fumaça, proxy reverso nginx, quatro dashboards Grafana provisionados e um Makefile de integração com vinte alvos.

Demais repositórios — parados em 2024

ESTADO DAS FERRAMENTAS DE CI		
PROBLEMA	ONDE	SITUAÇÃO
actions/checkout@v3, setup-node@v3, setup-python@v4	Front, Core, Action, Parser, CLI	DESCONTINUADAS
sonarcloud-github-action@master	Front, Core, Action	ACTION ARQUIVADA
Referência a 2023-2-MeasureSoftGram-Action@v2.2	Core	REPO RENOMEADO
Chaves sonar.projectKey divergentes	Todos	INCONSISTENTE
Etapa SonarCloud removida do CI	Parser	PERDA DE GATE

As chaves do Sonar divergem em três padrões: Service e Parser usam MeasureSoftGram-*, o CLI usa 2026.1-* e Core, Front e Action seguem em 2024.1-*. A organização também difere — o Service aponta para fga-eps-mds enquanto todos os demais usam fga-eps-mds-1.

NENHUM PORTÃO DE QUALIDADE É BLOQUEANTE

Os quatro arquivos codecov.yml — de CLI, Core, Parser e Service — são idênticos e marcam informational: true tanto em project quanto em patch. O .coveragerc do Service não define fail_under. Na prática, a cobertura de testes nunca reprova um pull request em um projeto cuja finalidade é justamente medir qualidade de software.

07 Dívida técnica acumulada

Front — a maior superfície, a mais desatualizada

São 18.936 linhas de TypeScript e TSX distribuídas em 72 arquivos de teste, sobre uma base cujas dependências pararam em 2022: Next.js 12.2.2, fora de suporte, axios@0.27.2 e Node 18 no CI, também em fim de vida. A biblioteca material-ui@0.20.2, anterior ao MUI, convive com @mui/material@5 — peso morto no bundle.

O README ainda se intitula “2024-1 MeasureSoftGram Frontend”, com badges do Sonar apontando para 2023-1- e link de produção para um domínio Vercel de 2024-1.

DIVERGÊNCIA NÃO EXPLICADA

O workflow de deploy do Service e a configuração nginx de produção roteiam um contêiner `front:3000` e registram que “o Front tem o seu próprio deploy + runner”. Existe, portanto, **um frontend em produção que não corresponde a este código**.

Action — parada há dois anos

- O manifesto `action.yml` declara `using: 'node16'`, runtime já removido pelo GitHub.
- O `package.json` ainda se chama `2023-1-measuresoftgram-action`, com campos `repository`, `bugs` e `homepage` apontando para o repositório antigo.
- `axios-mock-adapter`, `mock-fs` e `jest-sonar` estão em `dependencies` e não em `devDependencies` — vão para o pacote distribuído.
- O diretório `dist/` está versionado e foi gerado pela última vez em 10 de setembro de 2024.

Em contrapartida, os marcadores de dívida no código são poucos: apenas 14 ocorrências de `TODO`, `FIXME` ou `HACK` em cerca de 32 mil linhas de Python, treze delas concentradas no Service.

08 Pontos fortes

Vale registrar o que está sólido, porque não é pouco.

O núcleo matemático está bem testado

SÓLIDO

O Core alcança **92% de cobertura**, com código limpo e suíte que passa integralmente. O coração do modelo de qualidade — a parte mais difícil de reescrever — é a parte mais bem cuidada do projeto.

A modernização do Service é trabalho de nível profissional

SÓLIDO

Divisão de *settings* em dev, test e production com *docstrings* que explicam o porquê de cada decisão; actions fixadas por SHA; Grafana provisionado; deploy resiliente ao isolamento de rede do laboratório. É o padrão que deveria ser replicado nos demais repositórios.

A documentação nova é o melhor artefato do ciclo

SÓLIDO

O site em Docusaurus reúne 24 páginas estruturadas segundo o framework Diátaxis, com glossário, FAQ, diagramas de arquitetura em Mermaid, busca local e versionamento. Explica o ecossistema melhor do que qualquer README do projeto.

Disciplina de estilo mantida

SÓLIDO

Zero ocorrências de flake8 em cem por cento do código Python, e apenas 14 marcadores de dívida técnica em toda a base.

09 Plano de ação priorizado

Bloqueante **DIAS** correção imediata

- 1 Corrigir a fixture do Parser.** Duas *strings* em `expected_return_values.py`. O branch `develop` está vermelho.
- 2 Fechar o endpoint `GET /users/`.** Adicionar `DEFAULT_PERMISSION_CLASSES` no Service e marcar explicitamente os endpoints públicos.
- 3 Estabilizar a dependência do CLI.** Fixar `msgpack-core==1.5.3` ou atualizar as expectativas dos testes para a semântica correta do 1.5.4 — decisão de produto, não de infraestrutura.

Crítico **SEMANAS** integridade e segurança

- 4 **Recuperar o código do msgram-core 1.5.4** e versioná-lo no repositório Core. Enquanto isso não ocorrer, o modelo de qualidade permanece não auditável.
- 5 **Alinhar a versão do Core nos três consumidores.** O Service, em 1.4.9, é o mais atrasado.
- 6 **Atualizar o Django** da série 4.0 para 4.2 LTS, no mínimo, e cifrar o campo `github_access_token`.
- 7 **Habilitar TLS na borda** e ativar as flags de `production.py`, que já estão prontas e parametrizadas.

Estrutural

SEMESTRE

governança do ecossistema

- 8 **Decidir entre repositórios centrais ou de semestre.** Hoje a documentação afirma uma coisa e o trabalho recente faz outra.
- 9 **Replicar o padrão de CI do Service** nos outros seis repositórios: actions fixadas, `uv` e chaves do Sonar consistentes.
- 10 **Tornar bloqueante ao menos o portão de `patch` do Codecov.** Um projeto que mede qualidade não deveria manter os próprios portões em modo informativo.
- 11 **Atualizar ou aposentar Front e Action explicitamente**, e localizar o frontend que está de fato em produção.

10 Método e limitações

O diagnóstico distingue explicitamente o que foi verificado por execução do que resulta de leitura de código e configuração. As limitações abaixo delimitam o alcance das conclusões.

VERIFICADO POR EXECUÇÃO

- Suítes de teste de Core, Parser e CLI
- flake8 nos quatro repositórios Python
- Comparação dos pacotes 1.5.3 e 1.5.4 baixados do PyPI
- Histórico Git completo dos sete repositórios
- Rotas e permissões do Service por inspeção de código

NÃO VERIFICADO

- Suíte do Service — exige Postgres; Docker indisponível
- Suítes de Front e Action — dependências não instaladas
- Estado real da integração contínua no GitHub
- Repositórios AI e Plugin — ausentes do conjunto analisado
- Ambiente de produção em execução